

Lab1-ProbablyCoin-GirardW

September 30, 2026

1 Mini-Lab 1

Expected value and standard deviation of a coin-flip game

Submission Please simply show me this work in class, through Zoom, or convert the notebook to a PDF and send it via email.

This is one of the additional activities mentioned on the syllabus.

2 Expected value and standard deviation of a coin-flip game

2.0.1 Setup

You are playing a coin-flip game. If the coin lands heads, you win 2 dollars. If the coin lands tails, you lose 1 dolar. Assume the coin is fair (probability 0.5 each).

2.0.2 Your task

Fill in the missing parts below that simulate the game and quantify various statistical properties of your winnings. Missing parts are labelled with “TODO”

2.1 Part 1: simple statisitcs

```
[3]: import numpy as np

# number of coin flips
n_flips = 1000

# simulate coin flips: 1 for heads, 0 for tails
flips = np.random.randint(0, 2, size=n_flips)

# map outcomes to winnings: +2 for heads, -1 for tails
# TODO: write a Python function that does this.

def winning_map(flips):
    total = []

    for flip in flips:
        if flip == 0:
```

```

        total.append(-1)
    elif flip == 1:
        total.append(2)
    else:
        print("Invalid flip value receieved (outputs can only be 0 or 1)")

    return total

total = winning_map(flips)

# compute sample mean and std dev
mean_winnings = np.mean(total)
std_winnings = np.std(total)

print("Sample mean:", mean_winnings)
print("Sample std dev:", std_winnings)

```

Sample mean: 0.539

Sample std dev: 1.499492914288027

2.2 Part 2: law of large numbers

Now write code that runs the simulation for 10000 flips

Make two figures:

1. plot cumulative winnings vs number of flips
2. plot average winnings per flip vs number of flips. On the same plot, compare this number to the expected value of 0.5

You should find that the empirically measured average winnings converges to the expected value of 0.5. And the convergence is quantified by “standard deviation”/sqrt(flips). This is sometimes called the law of large numbers

```

[8]: import matplotlib.pyplot as plt

# sim 10,000 flips

n_flips = 10000
flips = np.random.randint(0, 2, size=n_flips)

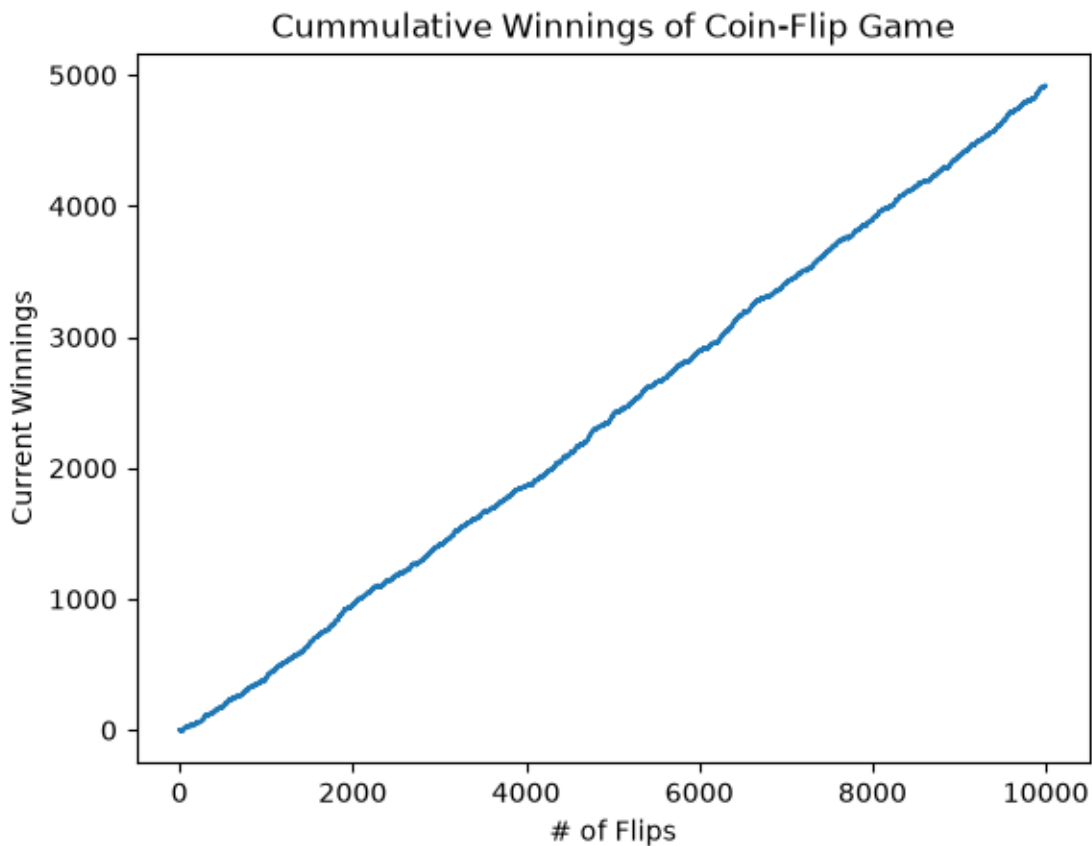
total_10k = winning_map(flips)

# calculate running total
running_total = np.cumsum(total_10k)
x = range(n_flips)

plt.step(x, running_total, where="post")
plt.title("Cumulative Winnings of Coin-Flip Game")

```

```
plt.xlabel("# of Flips")
plt.ylabel("Current Winnings")
plt.show()
```

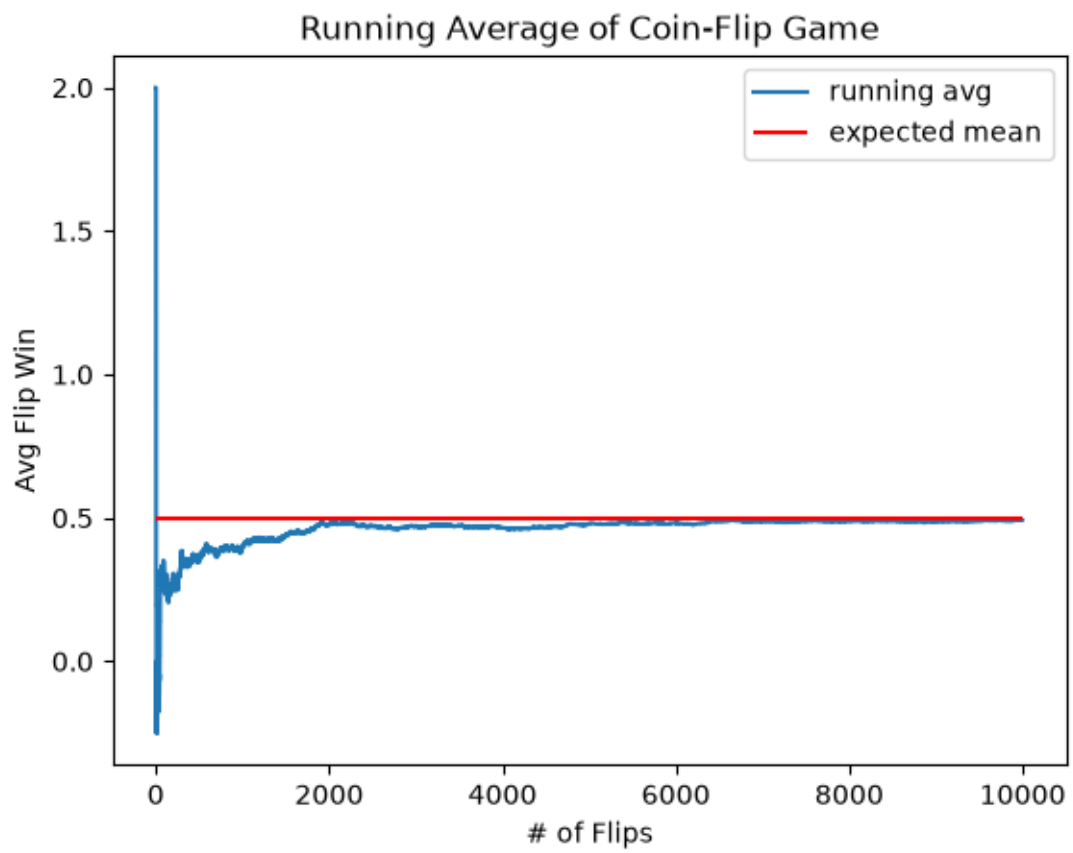


```
[21]: # compute running avg with a simple for loop

running_avg = []
total = 0
for i, flip in enumerate(total_10k):
    total += flip
    running_avg.append(total/(i+1))

plt.step(x, running_avg, where="post", label='running avg')
plt.hlines(y=0.5, xmin=0, xmax=10000, color='red', label='expected mean')
plt.title("Running Average of Coin-Flip Game")
plt.xlabel("# of Flips")
plt.ylabel("Avg Flip Win")
plt.legend()
```

```
plt.show()
```



```
[ ]:
```